DESIGN PATTERNS · BEHAVIORAL | GoF · 17 / 23 | 4-dive study card

Mediator

Define an object that encapsulates how a set of objects interact, so they no longer refer to each other explicitly.

DIVE 01 · 10 km — Airplane porthole — The big picture & contours
DIVE 02 · 1 km — Train window — Theses, examples, terms
DIVE 03 · 100 m — Park walk — How the pieces relate
DIVE 04 · 1 cm — Microscope — Exact wording & gotchas

Airplane porthole — DIVE 01 · ORIENTATION — ▼ 10 km

It turns a tangled mesh of colleagues talking directly into a star — every component reports to one hub, and the hub decides who reacts.

— THE CONTOURS

- **Centralizes interaction logic** — the "who talks to whom" lives in one place
- **Components know only the hub** — zero direct references between colleagues
- **Converts many-to-many to one** — N↔N coupling collapses into N↔1

— USE IT WHEN — GoF applicability

- **Objects communicate in complex, tangled ways** — and the resulting interdependencies are hard to follow
- **Reusing a component is hard** — because it refers to and talks with many others

Behavioral family: Mediator · Observer · Command · Strategy — Observer broadcasts to anonymous subscribers; Mediator centrally coordinates known colleagues.

DIVE DEEPER

Train window — DIVE 02 · KEY OBJECTS — ▼ 1 km

— THE THREE PARTS

- `Mediator` — the interface colleagues call — notify(sender, event)
- `ConcreteMediator` — coordinates colleagues, holds their references
- `Colleague` — a component that knows only its mediator

— IN THE WILD

- `Air-traffic control tower` — pilots talk to the tower, never plane-to-plane — GoF's own
- `MediatR (.NET)` — in-process CQRS — Send(request) dispatched to its handler
- `Redux / NgRx store` — components dispatch to one hub; it owns the transitions
- `DOM event delegation` — one parent listener routes events from many children

```
<> Colleague → Mediator.notify(sender, event) → ConcreteMediator routes → other Colleagues
```

GoF intent: "Define an object that encapsulates how a set of objects interact; it promotes loose coupling by keeping objects from referring to each other explicitly."

DIVE DEEPER

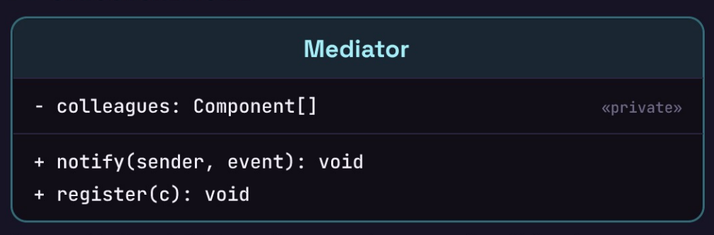
Park walk — DIVE 03 · CONNECTIONS — ▼ 100 m

Colleague acts (button clicked) → **notify(sender, event)** (tell the hub only) → **Mediator routes** (who should react?) → **✓ Colleagues update** (coordinated change)

— WHY IT FOLLOWS

- → **Single hub ⇒** interaction rules live in one class, not smeared across colleagues
- → **Colleagues are decoupled ⇒** each depends on the Mediator interface, not on its peers
- → **N↔N becomes N↔1 ⇒** add or swap a colleague without rewiring all the others

— STRUCTURE · UML



ConcreteMediator holds its -colleagues and routes each notify() to the ones that must react.

— WHAT IT BUYS YOU

- **Decouples colleagues —** components reference the mediator, never each other
- **Centralizes control —** scattered interaction logic collapses into one object
- **Reusable components —** a colleague drops into any mediator that speaks its protocol
- **Simplifies protocols —** many-to-many relations become one-to-many with the hub
- **Easier to vary interaction —** change the choreography by subclassing the mediator alone

Observer often implements the mediator↔colleague channel · Facade also fronts a subsystem but one-way & without coordination · a Singleton mediator is common but risky.

DIVE DEEPER

Microscope — DIVE 04 · DETAILS — ▼ 1 cm

Definition: "Define an object that encapsulates how a set of objects interact, promoting loose coupling by keeping them from referring to each other explicitly."
— GoF, "Design Patterns", 1994

— THE CODE

```
interface Mediator {
notify(sender: object, e: string): void;
}
class Dialog implements Mediator {
constructor(private ok: Button,
private box: Checkbox) {}
notify(s: object, e: string) {
if (e === "toggle") this.ok.enabled = …;
}                // colleagues call this
}
```

— VARIANTS

- **Direct mediator** — explicit notify() routing — classic GoF
- **Event bus** — pub/sub channel — looser, anonymous
- **MediatR** — request → single handler (in-proc CQRS)
- **Message broker** — out-of-process hub — Kafka, RabbitMQ
- **Observer-hybrid** — colleagues subscribe to the mediator
- **Command-mediator** — reified messages the hub dispatches

— COMPARE THE ALTERNATIVES

Property	Mediator	Observer	Facade	Event bus
Centralized hub	✓	×	✓	✓
Coordinates N↔N	✓	~	×	~
Decouples senders	✓	✓	~	✓
Sender knows target	×	×	✓	×
God-object risk	✓	×	~	~

Best fit → Mediator: coordinated widgets · Observer: broadcast · Facade: one-way entry · Bus: decoupled fan-out. ("~" = partial.)

— NUANCES & GOTCHAS

- **God-object risk** — all coordination logic piles into the mediator until it becomes a monolith that knows too much
- **N↔N → N↔1** — it trades pairwise coupling between colleagues for everyone's coupling to one central hub
- **vs Facade** — Facade simplifies a subsystem one-way; Mediator coordinates multi-way, peer-to-peer interaction
- **vs Observer** — Mediator is a centralized hub with routing logic; Observer is decentralized broadcast to subscribers
- **Testability win** — interaction rules sit in one class you can unit-test, instead of being smeared across components
- **Single point of failure** — centralizing flow concentrates risk; if the hub is wrong or down, every colleague is affected

— INTERVIEW PITFALLS

Q "Mediator vs Observer?"
Mediator centralizes who-talks-to-whom in one coordinator; Observer is decentralized one-to-many broadcast. They're often combined.

Q "Mediator vs Facade?"
Facade is a one-way simplified entry to a subsystem. Mediator manages multi-directional interaction between peers.

Q "The main downside?"
The mediator can become a god object — all interaction logic centralizes there and it grows hard to maintain.

Q "How does it reduce coupling?"
It turns N-to-N references between colleagues into N-to-1: each knows only the mediator, not its peers.

— WATCH OUT

⚠ **Anti-pattern:** When only two or three objects interact in a stable, simple way, a direct reference is clearer — a mediator adds an indirection layer that can quietly swell into a god object.

⚡ **Modern take:** In-process buses like MediatR and stores like Redux/NgRx are this pattern at framework scale — one dispatcher routes messages so features stay decoupled and independently testable.